

Лекция 5: функции как значения

Функциональное программирование на Haskell

Алексей Романов

13 сентября 2026 г.

МИЭТ

Функции как значения

- Как упоминалось в начале курса, одно из оснований ФП состоит в том, что функции могут использоваться как значения.
- В Haskell можно выразиться сильнее:

Функции как значения

- Как упоминалось в начале курса, одно из оснований ФП состоит в том, что функции могут использоваться как значения.
- В Haskell можно выразиться сильнее:
- Функции это и есть просто значения, тип которых имеет форму `ТипПараметра -> ТипРезультата` для каких-то `ТипПараметра` и `ТипРезультата`.

Функции как значения

- Как упоминалось в начале курса, одно из оснований ФП состоит в том, что функции могут использоваться как значения.
- В Haskell можно выразиться сильнее:
- Функции это и есть просто значения, тип которых имеет форму `ТипПараметра -> ТипРезультата` для каких-то `ТипПараметра` и `ТипРезультата`.
- Мы уже видели примеры этого в равноправии функций и переменных.

Функции как значения

- Как упоминалось в начале курса, одно из оснований ФП состоит в том, что функции могут использоваться как значения.
- В Haskell можно выразиться сильнее:
- Функции это и есть просто значения, тип которых имеет форму `ТипПараметра -> ТипРезультата` для каких-то `ТипПараметра` и `ТипРезультата`.
- Мы уже видели примеры этого в равноправии функций и **других** переменных.

Функции высших порядков

- Функции могут принимать на вход функции.
- Тривиальный пример:

```
import Data.Char (isLetter, toLower)
```

```
foo :: (Char -> Bool) -> Bool
```

```
foo f = f 'a'
```

```
ghci> foo isLetter
```

Функции высших порядков

- Функции могут принимать на вход функции.
- Тривиальный пример:

```
import Data.Char (isLetter, toLower)
```

```
foo :: (Char -> Bool) -> Bool
```

```
foo f = f 'a'
```

```
ghci> foo isLetter
```

```
True
```

- Скобки вокруг типа параметра здесь необходимы.

Функции высших порядков

- Функции могут принимать на вход функции.
- Тривиальный пример:

```
import Data.Char (isLetter, toLower)
```

```
foo :: (Char -> Bool) -> Bool
```

```
foo f = f 'a'
```

```
ghci> foo isLetter
```

```
True
```

- Скобки вокруг типа параметра здесь необходимы.
- В других языках *функциями высших порядков (ФВП)* называют функции, аргументы или результаты которых тоже функции.

Функции высших порядков

- Функции могут принимать на вход функции.
- Тривиальный пример:

```
import Data.Char (isLetter, toLower)
```

```
foo :: (Char -> Bool) -> Bool
```

```
foo f = f 'a'
```

```
ghci> foo isLetter
```

```
True
```

- Скобки вокруг типа параметра здесь необходимы.
- В других языках *функциями высших порядков (ФВП)* называют функции, аргументы или результаты которых тоже функции.
- Функция второго порядка принимает или возвращает обычную функцию (первого порядка) и так далее.

Функции высших порядков

- Функции могут принимать на вход функции.
- Тривиальный пример:

```
import Data.Char (isLetter, toLower)
```

```
foo :: (Char -> Bool) -> Bool
```

```
foo f = f 'a'
```

```
ghci> foo isLetter
```

```
True
```

- Скобки вокруг типа параметра здесь необходимы.
- В других языках *функциями высших порядков (ФВП)* называют функции, аргументы или результаты которых тоже функции.
- Функция второго порядка принимает или возвращает обычную функцию (первого порядка) и так далее.
- В этой лекции ФВП называем только функции, принимающие функции. Почему — увидим позже.

Лямбда-выражения

- В `foo` с прошлого слайда можем также передать свою новую функцию, определив её локально:

```
bar = foo isCyrillic where  
    isCyrillic c = let lc = toLower c  
                  in 'a' <= lc && lc <= 'я'
```

Лямбда-выражения

- В `foo` с прошлого слайда можем также передать свою новую функцию, определив её локально:

```
bar = foo isCyrillic where  
    isCyrillic c = let lc = toLower c  
                  in 'a' <= lc && lc <= 'я'
```

- Но имя этой функции на самом деле не нужно.
- Как и вообще функциям, которые создаются только как аргументы для других (или как результаты).

Лямбда-выражения

- В `foo` с прошлого слайда можем также передать свою новую функцию, определив её локально:

```
bar = foo isCyrillic where
    isCyrillic c = let lc = toLower c
                   in 'a' <= lc && lc <= 'я'
```

- Но имя этой функции на самом деле не нужно.
- Как и вообще функциям, которые создаются только как аргументы для других (или как результаты).
- Вместо этого зададим её через *лямбда-выражение*

```
bar = foo (\c -> let lc = toLower c
                 in 'a' <= lc && lc <= 'я')
```

Лямбда-выражения

- В `foo` с прошлого слайда можем также передать свою новую функцию, определив её локально:

```
bar = foo isCyrillic where
    isCyrillic c = let lc = toLower c
                   in 'a' <= lc && lc <= 'я'
```

- Но имя этой функции на самом деле не нужно.
- Как и вообще функциям, которые создаются только как аргументы для других (или как результаты).
- Вместо этого зададим её через *лямбда-выражение*

```
bar = foo (\c -> let lc = toLower c
                  in 'a' <= lc && lc <= 'я')
```

- Кстати, почему это определение неверно?

Лямбда-выражения

- В `foo` с прошлого слайда можем также передать свою новую функцию, определив её локально:

```
bar = foo isCyrillic where
    isCyrillic c = let lc = toLower c
                   in 'a' <= lc && lc <= 'я'
```

- Но имя этой функции на самом деле не нужно.
- Как и вообще функциям, которые создаются только как аргументы для других (или как результаты).
- Вместо этого зададим её через *лямбда-выражение*

```
bar = foo (\c -> let lc = toLower c
                  in 'a' <= lc && lc <= 'я')
```

- Кстати, почему это определение неверно?
- Как ни странно, в Юникоде `'ё' > 'я'`.
- И не все буквы кириллицы используются в русском.

Лямбда-выражения

- Вообще, два определения
функция образец = результат

функция = \образец -> результат
эквивалентны.

Лямбда-выражения

- Вообще, два определения
функция образец = результат

функция = \образец -> результат
эквивалентны.

- Одно исключение: для второго может быть выведен менее общий тип.

Лямбда-выражения

- Вообще, два определения
`функция образец = результат`

`функция = \образец -> результат`
эквивалентны.

- Одно исключение: **для второго может быть выведен менее общий тип.**
- Лямбда-выражение для функции с несколькими параметрами пишется
`\образец1 ... образецN -> результат`

Лямбда-выражения

- Вообще, два определения
`функция` образец = результат

`функция` = \образец -> результат
эквивалентны.

- Одно исключение: **для второго может быть выведен менее общий тип.**
- Лямбда-выражение для функции с несколькими параметрами пишется
`\образец1 ... образецN -> результат`
- Например,

```
import Data.List (sortBy)
```

```
sortBy (\x y -> compare y x) list
```

- Что делает это выражение?

Лямбда-выражения

- Вообще, два определения
`функция` образец = результат

`функция` = \образец -> результат
эквивалентны.

- Одно исключение: для второго может быть выведен менее общий тип.
- Лямбда-выражение для функции с несколькими параметрами пишется
`\образец1 ... образецN -> результат`
- Например,

```
import Data.List (sortBy)
```

```
sortBy (\x y -> compare y x) list
```

- Что делает это выражение?
- Сортирует список по убыванию.

Лямбда-выражения и **case**

- Если в обычном определении функции несколько уравнений, например

```
not' True  = False
```

```
not' False = True
```

то в лямбда-выражении придётся использовать **case**:

Лямбда-выражения и **case**

- Если в обычном определении функции несколько уравнений, например

```
not' True  = False  
not' False = True
```

то в лямбда-выражении придётся использовать **case**:

```
not' = \x -> case x of  
    True  -> False  
    False -> True
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

`( $\$$ ) :: (a -> b) -> a -> b`

`f $ x =`

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
( $\$$ ) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

`( $\$$ ) :: (a -> b) -> a -> b`

`f $ x = f x`

`( $\cdot$ ) :: (b -> c) -> (a -> b) -> (a -> c)`

`g . f =`

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

```
g . f = \x ->
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

```
g . f = \x -> g (f x)
```

```
flip :: (a -> b -> c) -> (b -> a -> c)
```

```
flip f =
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

```
g . f = \x -> g (f x)
```

```
flip :: (a -> b -> c) -> (b -> a -> c)
```

```
flip f = \y x ->
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

```
g . f = \x -> g (f x)
```

```
flip :: (a -> b -> c) -> (b -> a -> c)
```

```
flip f = \y x -> f x y
```

- И ещё две в **Data.Function**:

```
(&) :: a -> (a -> b) -> b
```

```
x & f =
```

Несколько стандартных ФВП

- В **Prelude** есть три функции второго порядка, которые очень часто встречаются в коде Haskell:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
(.) :: (b -> c) -> (a -> b) -> (a -> c)
```

```
g . f = \x -> g (f x)
```

```
flip :: (a -> b -> c) -> (b -> a -> c)
```

```
flip f = \y x -> f x y
```

- И ещё две в **Data.Function**:

```
(&) :: a -> (a -> b) -> b
```

```
x & f = f x    -- то есть (&) = flip ($)
```

```
on :: (b -> b -> c) -> (a -> b) ->
```

```
    (a -> a -> c)
```

```
on op f x y = op (f x) (f y)
```

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать $f(x)$ вместо $f \ x$?

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать $f \$ x$ вместо $f x$?
- Этот оператор имеет минимальный возможный приоритет, так что $f (x + y)$ можно записать как $f \$ x + y$.

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать $f \$ x$ вместо $f x$?
- Этот оператор имеет минимальный возможный приоритет, так что $f (x + y)$ можно записать как $f \$ x + y$.
- И он правоассоциативен, так что $f (g (h x))$ можно записать как $f \$ g \$ h x$.

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать $f \$ x$ вместо $f x$?
- Этот оператор имеет минимальный возможный приоритет, так что $f (x + y)$ можно записать как $f \$ x + y$.
- И он правоассоциативен, так что $f (g (h x))$ можно записать как $f \$ g \$ h x$.
- Но более принято $f . g . h \$ x$.

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать $f \$ x$ вместо $f x$?
- Этот оператор имеет минимальный возможный приоритет, так что $f (x + y)$ можно записать как $f \$ x + y$.
- И он правоассоциативен, так что $f (g (h x))$ можно записать как $f \$ g \$ h x$.
- Но более принято $f . g . h \$ x$.
- $.$ тоже правоассоциативен, но уже с максимальным приоритетом.

Избавление от скобок с помощью \$ и .

- Казалось бы, в чём смысл \$: зачем писать `f $ x` вместо `f x`?
- Этот оператор имеет минимальный возможный приоритет, так что `f (x + y)` можно записать как `f $ x + y`.
- И он правоассоциативен, так что `f (g (h x))` можно записать как `f $ g $ h x`.
- Но более принято `f . g . h $ x`.
- `.` тоже правоассоциативен, но уже с максимальным приоритетом.
- Пока, наверное, проще читать и писать код со скобками, но стандартный стиль Haskell предпочитает их избегать.
- Только не перестарайтесь!

Правда о функциях многих переменных

- Настала пора раскрыть тайну функций многих переменных в Haskell:

Правда о функциях многих переменных

- Настала пора раскрыть тайну функций многих переменных в Haskell:
- Их не существует.

Правда о функциях многих переменных

- Настала пора раскрыть тайну функций многих переменных в Haskell:
- Их не существует.
- `->` — правоассоциативный оператор, так что `Тип1 -> Тип2 -> Тип3` это на самом деле `Тип1 -> (Тип2 -> Тип3)`: функция, возвращающая функцию.

Правда о функциях многих переменных

- Настала пора раскрыть тайну функций многих переменных в Haskell:
- Их не существует.
- `->` — правоассоциативный оператор, так что `Тип1 -> Тип2 -> Тип3` это на самом деле `Тип1 -> (Тип2 -> Тип3)`: функция, возвращающая функцию.
- `\x у -> результат` это сокращение для `\x -> \у -> результат`, а со скобками `\x -> (\у -> результат)`.

Правда о функциях многих переменных

- Настала пора раскрыть тайну функций многих переменных в Haskell:
- Их не существует.
- `->` — правоассоциативный оператор, так что `Тип1 -> Тип2 -> Тип3` это на самом деле `Тип1 -> (Тип2 -> Тип3)`: функция, возвращающая функцию.
- `\x у -> результат` это сокращение для `\x -> \у -> результат`, а со скобками `\x -> (\у -> результат)`.
- Применение функций (не `$`, а пробел), наоборот, левоассоциативно. То есть `f x у` читается как `(f x) у`.

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.

Каррирование

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.
- В Haskell тоже можно было бы писать
`foo :: (Int, Int) -> Int`
`foo (x, y) = x + y`
для определения функций и `foo (1, 2)` для вызова.

Каррирование

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.
- В Haskell тоже можно было бы писать

```
foo :: (Int, Int) -> Int
foo (x, y) = x + y
```

для определения функций и `foo (1, 2)` для вызова.
- Но Haskell здесь следует традиции λ -исчисления.

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.
- В Haskell тоже можно было бы писать

```
foo :: (Int, Int) -> Int
foo (x, y) = x + y
```

для определения функций и `foo (1, 2)` для вызова.
- Но Haskell здесь следует традиции λ -исчисления.
- Эти подходы эквивалентны, так как множества $A \times B \rightarrow C$ и $A \rightarrow (B \rightarrow C)$ всегда изоморфны

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.
- В Haskell тоже можно было бы писать

```
foo :: (Int, Int) -> Int
foo (x, y) = x + y
```

для определения функций и `foo (1, 2)` для вызова.
- Но Haskell здесь следует традиции λ -исчисления.
- Эти подходы эквивалентны, так как множества $A \times B \rightarrow C$ и $A \rightarrow (B \rightarrow C)$ всегда изоморфны ($C^{A \cdot B} = (C^B)^A$).
- В Haskell этот изоморфизм реализуют функции

- Обычно в математике для сведения функций нескольких аргументов к функциям одного аргумента используется декартово произведение: $A \times B \rightarrow C$, а не $A \rightarrow (B \rightarrow C)$.
- В Haskell тоже можно было бы писать

```
foo :: (Int, Int) -> Int
foo (x, y) = x + y
```

для определения функций и `foo (1, 2)` для вызова.
- Но Haskell здесь следует традиции λ -исчисления.
- Эти подходы эквивалентны, так как множества $A \times B \rightarrow C$ и $A \rightarrow (B \rightarrow C)$ всегда изоморфны ($C^{A \cdot B} = (C^B)^A$).
- В Haskell этот изоморфизм реализуют функции

```
curry :: ((a, b) -> c) -> a -> b -> c
uncurry :: (a -> b -> c) -> (a, b) -> c
```

Частичное применение

- Преимущество каррированных функций в том, что естественным образом появляется частичное применение.
- То есть мы можем применить функцию «двух аргументов» только к первому и останется функция одного аргумента:

```
ghci> :t sortBy  
sortBy :: (a -> a -> Ordering) -> [a] -> [a]  
ghci> :t sortBy (flip compare)
```

Частичное применение

- Преимущество каррированных функций в том, что естественным образом появляется частичное применение.
- То есть мы можем применить функцию «двух аргументов» только к первому и останется функция одного аргумента:

```
ghci> :t sortBy
sortBy :: (a -> a -> Ordering) -> [a] -> [a]
ghci> :t sortBy (flip compare)
sortBy (flip compare) :: Ord a => [a] -> [a]
```

Частичное применение

- Преимущество каррированных функций в том, что естественным образом появляется частичное применение.
- То есть мы можем применить функцию «двух аргументов» только к первому и останется функция одного аргумента:

```
ghci> :t sortBy
sortBy :: (a -> a -> Ordering) -> [a] -> [a]
ghci> :t sortBy (flip compare)
sortBy (flip compare) :: Ord a => [a] -> [a]
```

- Заметьте, что здесь частичное применение в двух местах:

Частичное применение

- Преимущество каррированных функций в том, что естественным образом появляется частичное применение.
- То есть мы можем применить функцию «двух аргументов» только к первому и останется функция одного аргумента:

```
ghci> :t sortBy
sortBy :: (a -> a -> Ordering) -> [a] -> [a]
ghci> :t sortBy (flip compare)
sortBy (flip compare) :: Ord a => [a] -> [a]
```

- Заметьте, что здесь частичное применение в двух местах: `flip` можно теперь рассматривать как функцию трёх аргументов!

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1
```

```
(+) 1 :: Num a => a -> a
```

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1  
(+) 1 :: Num a => a -> a
```

- А ко второму? Можно использовать лямбду

```
\x -> x / 2
```

или `flip`

```
flip (/) 2
```

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1  
(+) 1 :: Num a => a -> a
```

- А ко второму? Можно использовать лямбду

```
\x -> x / 2
```

или `flip`

```
flip (/) 2
```

- Но есть специальный синтаксис (arg op) и (op arg):

```
ghci> (1 `div`) 2
```

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1  
(+) 1 :: Num a => a -> a
```

- А ко второму? Можно использовать лямбду

```
\x -> x / 2
```

или `flip`

```
flip (/) 2
```

- Но есть специальный синтаксис (arg op) и (op arg):

```
ghci> (1 `div`) 2
```

```
0
```

```
ghci> (/ 2) 4
```

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1  
(+) 1 :: Num a => a -> a
```

- А ко второму? Можно использовать лямбду

```
\x -> x / 2
```

или `flip`

```
flip (/) 2
```

- Но есть специальный синтаксис (`arg op`) и (`op arg`):

```
ghci> (1 `div`) 2
```

```
0
```

```
ghci> (/ 2) 4
```

```
2.0
```

Сечения операторов

- Для применения бинарного оператора только к первому аргументу можно использовать его префиксную форму:

```
ghci> :t (+) 1  
(+) 1 :: Num a => a -> a
```

- А ко второму? Можно использовать лямбду

```
\x -> x / 2
```

или `flip`

```
flip (/) 2
```

- Но есть специальный синтаксис (arg op) и (op arg):

```
ghci> (1 `div`) 2
```

```
0
```

```
ghci> (/ 2) 4
```

```
2.0
```

- Ловушка: `(- 2)` — это число `-2`, а не сечение. Для вычитания есть `subtract 2`.

Сечения кортежей

- Конструкторы кортежей тоже можно применять частично (**TupleSections**, входит в GHC2021):

```
ghci> (, "I", , "Love", True) 1 False
```

- Конструкторы кортежей тоже можно применять частично (**TupleSections**, входит в GHC2021):

```
ghci> (, "I", , "Love", True) 1 False  
(1,"I",False,"Love",True)  
ghci> :t (, "I", , "Love", True)
```

- Конструкторы кортежей тоже можно применять частично (`TupleSections`, входит в GHC2021):

```
ghci> (, "I", , "Love", True) 1 False
(1,"I",False,"Love",True)
ghci> :t (, "I", , "Love", True)
(, "I", , "Love", True)
  :: t1 -> t2 -> (t1, String, t2, String, Bool)
```

η -эквивалентность (сокращение аргументов)

- Рассмотрим два определения

`foo' x = foo x`

-- или $foo' = \lambda x \rightarrow foo\ x$

- Применение обоих определений к одному аргументу задаёт одно и то же вычисление.

η -эквивалентность (сокращение аргументов)

- Рассмотрим два определения

`foo' x = foo x`

-- или `foo' = \x -> foo x`

- Применение обоих определений к одному аргументу задаёт одно и то же вычисление.
- Поэтому обычно можно сократить аргумент:

`foo' = foo`

η -эквивалентность (сокращение аргументов)

- Рассмотрим два определения

`foo' x = foo x`

-- или `foo' = \x -> foo x`

- Применение обоих определений к одному аргументу задаёт одно и то же вычисление.
- Поэтому обычно можно сократить аргумент:

`foo' = foo`

- Оговорка: удаление лямбды может изменить поведение при использовании `seq` (подробнее в лекции о ленивости).
- Можно сокращать и несколько аргументов в конце:

`foo' x y z w = foo (y + x) z w`

эквивалентно

`foo' x y = foo (y + x)`

η -эквивалентность (сокращение аргументов)

- И когда само определение такой формы не имеет, но может быть к ней преобразовано

```
root4 :: Floating a => a -> a
```

```
root4 x = sqrt (sqrt x)
```

можно переписать (сохраняя сигнатуру) как

```
root4 x = (sqrt . sqrt) x
```

```
root4 = sqrt . sqrt
```

- Ещё пример:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
($) f x = f x
```

η -эквивалентность (сокращение аргументов)

- И когда само определение такой формы не имеет, но может быть к ней преобразовано

```
root4 :: Floating a => a -> a
```

```
root4 x = sqrt (sqrt x)
```

можно переписать (сохраняя сигнатуру) как

```
root4 x = (sqrt . sqrt) x
```

```
root4 = sqrt . sqrt
```

- Ещё пример:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
($) f x = f x
```

```
($) f = f
```

η -эквивалентность (сокращение аргументов)

- И когда само определение такой формы не имеет, но может быть к ней преобразовано

```
root4 :: Floating a => a -> a
```

```
root4 x = sqrt (sqrt x)
```

можно переписать (сохраняя сигнатуру) как

```
root4 x = (sqrt . sqrt) x
```

```
root4 = sqrt . sqrt
```

- Ещё пример:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
($) f x = f x
```

```
($) f = f
```

```
($) f = id f
```

η -эквивалентность (сокращение аргументов)

- И когда само определение такой формы не имеет, но может быть к ней преобразовано

```
root4 :: Floating a => a -> a
```

```
root4 x = sqrt (sqrt x)
```

можно переписать (сохраняя сигнатуру) как

```
root4 x = (sqrt . sqrt) x
```

```
root4 = sqrt . sqrt
```

- Ещё пример:

```
($) :: (a -> b) -> a -> b
```

```
f $ x = f x
```

```
($) f x = f x
```

```
($) f = f
```

```
($) f = id f
```

```
($) = id
```

Бесточечный стиль

- Композиция и другие операции позволяют определять функции без явного указания аргументов.
- Это называется *бесточечным стилем* (переменные рассматриваются как точки в пространстве значений).

Бесточечный стиль

- Композиция и другие операции позволяют определять функции без явного указания аргументов.
- Это называется *бесточечным стилем* (переменные рассматриваются как точки в пространстве значений).
- Многие выражения имеют бесточечный эквивалент.
- На [Haskell Wiki](#) есть инструменты для перевода между стилями.
- Опять же, не перестарайтесь:

```
> pl \x y -> compare (f x) (f y)  
(. f) . compare . f
```

Бесточечный стиль

- Композиция и другие операции позволяют определять функции без явного указания аргументов.
- Это называется *бесточечным стилем* (переменные рассматриваются как точки в пространстве значений).
- Многие выражения имеют бесточечный эквивалент.
- На [Haskell Wiki](#) есть инструменты для перевода между стилями.
- Опять же, не перестарайтесь:

```
> pl \x y -> compare (f x) (f y)  
(. f) . compare . f
```
- Яснее выразить тот же смысл через `on`:

```
import Data.Function (on)  
compare `on` f
```
- В языках семейств Forth и APL бесточечный стиль является основным.

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo
```

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) =
```

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x ->
```

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x -> ?
```

Типы переменных:

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x -> ?
```

Типы переменных:

```
f :: a -> b, g :: b -> c,
```

Программирование, направляемое типами

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x -> ?
```

Типы переменных:

```
f :: a -> b, g :: b -> c, x :: a, ? :: c
```

Можно увидеть, что ? может быть **g** ??, где ?? ::

Программирование, направляемое типами

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x -> ?
```

Типы переменных:

```
f :: a -> b, g :: b -> c, x :: a, ? :: c
```

Можно увидеть, что ? может быть `g ??`, где `?? :: b`
=

Программирование, направляемое типами

- При реализации полиморфных функций очень часто их тип подсказывает реализацию.
- Пример:

```
foo :: (a -> b, b -> c) -> (a -> c)
```

Какую часть реализации можно написать сразу?

```
foo (f, g) = \x -> ?
```

Типы переменных:

```
f :: a -> b, g :: b -> c, x :: a, ? :: c
```

Можно увидеть, что ? может быть `g ??`, где `?? :: b = f x`.

Типизированные дыры в коде

- Эти рассуждения не обязательно делать вручную.
- Если в выражении (а не в образце) использовать дыру `_` (или `_название`; дыр может быть несколько), то увидим ожидаемый в этом месте тип.
- `foo :: (a -> b, b -> c) -> (a -> c)`
`foo (f, g) = \x -> g _`
выдаст сообщение
 - Found hole: `_ :: b`
Where: 'b' is a rigid type variable bound by
the type signature for foo
 - Relevant bindings include
`x :: a`, `g :: b -> c`, `f :: a -> b`, ...
- *Rigid* (жёсткая) переменная типа — из сигнатуры: тип выбирает вызывающий, и внутри определения её нельзя заменить конкретным. Та же фраза будет в ошибке, если вернуть `Int` вместо `a`.